

```
pass
passDemo ()
```

Example 2:

```
marks = 80
if marks >= 75:
    pass
```

Counter function in collections module

The *collections* module of Python has different types of containers. *Counter*, one of the containers in the *collections* module, is a *dict* subclass that provides an efficient way to count objects. The *keys* component of the dictionary stores the object, whose occurrence has to be counted, and the *values* component of the dictionary holds the object's count, that is, its frequency. *Counter* takes an iterable list as an argument and returns a *dict* object containing each distinct item in the iterable list and its frequency as a key-value pair. For example:

```
>>> from collections import Counter
>>> alphabets = [ 'a', 'b', 'c', 'd', 'b', 'a', 'b', 'c',
' a', 'b', 'c' ]
>>> num_alphabets = Counter ( alphabets )
>>> print ( num_alphabets )
Counter({'b': 4, 'a': 3, 'c': 3, 'd': 1})
```

Notice that the key-value pairs are ordered as per the descending order of the count of the occurrences in the iterable. The *Counter* can also be used to merge two dictionaries. The keys are preserved and the values of matching keys are added. For example:

```
>>> from collections import Counter
>>> marks1 = { 'Ganesh': 90, 'Siva Sundar': 88, 'Gunasekhar':
75, 'Santosh': 72 }
>>> marks2 = { 'Ganesh': 90, 'Siva Sundar': 88, 'Gunasekhar':
75, 'Santosh': 72, 'Shashank':78 }
>>> finalmarks = Counter( marks1 ) + Counter ( marks2 )
>>> print ( finalmarks )
Counter({'Ganesh': 180, 'Siva Sundar': 176, 'Gunasekhar':
150, 'Santosh': 144, 'Shashank': 78})
```

Expression evaluation using the eval function

The built-in *eval* function accepts a string as an argument. If the string is an expression, then the expression is evaluated. This is useful for evaluating any Python expression that comes as an input dynamically. For example:

```
>>> x = 7
```

```
>>> y= 45
>>> eval('x * y')
315
```

```
>>> eval ('2 ** 8')
256
```

```
>>> z = eval('x * y + 4')
>>> print (z)
319
```

Factorial

Factorial of a non-negative integer 'n' is the product of all positive integers less than or equal to 'n', but greater than or equal to 1. The *math* module of Python has a function named *factorial* to compute the factorial of an integer passed as an argument to the function.

```
>>> import math
>>> num = 9
>>> fact = math.factorial ( num )
>>> print ( fact )
362880
```

Playing sound notifications in Python

If you need to notify the user of the sound of some event, you can use the *chime* module to generate the standard sounds.

```
import chime
chime.info ()
chime.success ()
chime.error ()
chime.warning ()
```

The above tips will help programmers code faster, create more efficient and concise code, and will increase their productivity. The use of powerful features like *zip*, *Counter* and list comprehension makes the code concise and easy to understand. Happy Pythoning! 

References

- <https://www.tiobe.com/tiobe-index/>
- <https://pypl.github.io/PYPL.html>
- <https://www.opensourceforu.com/2021/04/python-a-few-useful-tips-for-beginners/>

By: Sathyanarayanan S.

The author is working with Sri Sathya Sai University for Human Excellence, Gulbarga. He has more than 30 years of experience in systems management and in teaching IT courses. He is an enthusiastic promoter of FOSS.